

MASTERCLASS D'INGÉNIERIE PÉDAGOGIQUE

ROADMAP D'APPRENTISSAGE DÉVELOPPEMENT D'AGENTS IA

Le plan directeur ultime pour maîtriser la conception, l'orchestration, l'évaluation et l'optimisation des Agents Intelligents et Systèmes Multi-Agents de débutant à expert.

 Version
2026

 Approche Agentique &
Pragmatique

 8 Phases
Structurées

PHASE 0 : FONDATIONS

1. Nom de la phase et durée estimée

- Nom : Fondations Techniques et Protocolaires
- Durée estimée : 2 semaines

2. Objectif global de la phase

- Maîtriser les prérequis indispensables en programmation, manipulation de données et environnements de développement nécessaires pour concevoir des applications interagissant avec l'IA.

3. Liste des compétences clés à acquérir

- Écrire et structurer du code en Python ou JavaScript/TypeScript.
- Manipuler des structures de données JSON et consommer des API REST via HTTP.
- Gérer le versionnage de code avec Git et utiliser le terminal en ligne de commande.
- Configurer un environnement virtuel et installer des dépendances de paquets.

4. Liste des concepts théoriques à comprendre

- Programmation synchrone et asynchrone (bases).
- Requêtes et réponses HTTP (méthodes, en-têtes, codes de statut).
- Sérialisation et désérialisation de données (JSON).
- Gestion des flux de contrôle et structures de données fondamentales (listes, dictionnaires, objets).

5. Outils et technologies à découvrir

- Python / Node.js (Runtime)
- Git & GitHub (Alternatives : GitLab, Gitea)
- Terminal (Bash / PowerShell / Zsh)
- Postman / cURL / Bruno (Alternatives gratuites pour tester les API)

- VS Code / Cursor (Éditeurs de code)

6. Prérequis pour aborder la phase

- Aucun prérequis technique particulier (niveau débutant absolu).

7. Types d'exercices à réaliser

- Script d'automatisation simple de manipulation de fichiers locaux en script.
- Consommation d'une API publique ouverte pour récupérer et filtrer des données JSON.
- Initialisation d'un dépôt Git, création de branches et commit de modifications.
- Installation de paquets tiers via un gestionnaire de paquets (pip ou npm).

8. Projet capstone de fin de phase

- Création d'un script en ligne de commande (CLI) qui interroge une API météo publique, formate les données reçues en JSON propre, et enregistre un rapport localement tout en sauvegardant l'historique dans un dépôt Git.

9. Critères de validation pour passer à la phase suivante

- Capacité à écrire un script fonctionnel qui interagit avec une API externe sans assistance.
- Maîtrise des commandes Git de base (clone, add, commit, push).
- Compréhension de la structure d'un objet JSON et capacité à y extraire des valeurs imbriquées.

10. Ressources recommandées

- Documentation officielle des langages (Python.org / Node.js).
- Cours interactifs en ligne sur les bases de Git et du terminal.
- Guides sur le protocole HTTP et les principes des API REST.

PHASE 1 : INTRODUCTION AUX LLMS

1. Nom de la phase et durée estimée

- Nom : Introduction aux Grands Modèles de Langage (LLMs)
- Durée estimée : 2 semaines

2. Objectif global de la phase

- Comprendre le fonctionnement fondamental des LLMs, maîtriser l'art du prompting programmatique et intégrer des modèles via leurs API respectives en tenant compte des aspects économiques.

3. Liste des compétences clés à acquérir

- Rédiger des prompts structurés et efficaces (few-shot, chain-of-thought, system prompts).
- Consommer les API de fournisseurs de LLMs (OpenAI, Anthropic, Google Gemini, ou équivalents open source).
- Gérer la consommation des tokens et estimer les coûts associés aux requêtes.
- Paramétrer les hyperparamètres d'inférence (température, top_p, max_tokens).

4. Liste des concepts théoriques à comprendre

- Tokens et tokenisation.
- Fenêtre de contexte (Context Window).
- Température et déterminisme dans la génération.
- System Prompt vs User Prompt vs Assistant Message.
- Modèles tarifaires par token (input/output).

5. Outils et technologies à découvrir

- OpenAI API / Anthropic API / Google Gemini API.
- OpenRouter / Groq (Alternatives pour tester différents modèles à moindre coût).
- LiteLLM (Abstraction pour appeler plusieurs fournisseurs).
- Tokenizer playground (outils de visualisation de tokens).

6. Prérequis pour aborder la phase

- Validation de la Phase 0 (maîtrise des API REST et de Python/JS).

7. Types d'exercices à réaliser

- Script d'appel d'API LLM simple avec passage d'un prompt système.

- Implémentation d'une fonction de calcul et de limitation du nombre de tokens avant envoi.
- Test comparatif des réponses d'un même prompt avec différentes températures.
- Rédaction d'un prompt complexe utilisant la technique du "Chain-of-Thought".

8. Projet capstone de fin de phase

- Développement d'un assistant de résumé de texte automatisé par API, capable de prendre un long document brut, de respecter une limite de tokens stricte, d'ajuster dynamiquement les paramètres d'inférence selon le type de document, et de journaliser le coût exact de chaque requête.

9. Critères de validation pour passer à la phase suivante

- Capacité à intégrer une API LLM dans un script propre avec gestion des erreurs (rate limits, timeouts).
- Maîtrise des concepts de base du prompting programmatique.
- Compréhension claire de l'impact financier et technique des tokens.

10. Ressources recommandées

- Documentation officielle des fournisseurs de LLMs (OpenAI, Anthropic).
- Guides de prompting (OpenAI Cookbook, Anthropic Prompt Engineering Interactive Tutorial).
- Articles de recherche fondateurs sur l'architecture Transformer (vulgarisation).

PHASE 2 : PREMIER AGENT SIMPLE

1. Nom de la phase et durée estimée

- Nom : Conception et Implémentation d'un Agent Simple (ReAct)
- Durée estimée : 3 semaines

2. Objectif global de la phase

- Transformer un simple LLM en un agent autonome capable de raisonner, de planifier des étapes et d'appeler des fonctions externes (Tool Calling) pour résoudre des tâches.

3. Liste des compétences clés à acquérir

- Structurer une boucle de raisonnement (modèle de type Perception-Raisonnement-Action).
- Définir, documenter et exposer des fonctions (tools) exploitables par un LLM.
- Intercepter les requêtes de tool calling du modèle et exécuter le code local correspondant.
- Réinjecter les résultats d'exécution des outils dans le contexte du LLM.

4. Liste des concepts théoriques à comprendre

- Le pattern ReAct (Reason + Act).
- Le Function Calling / Tool Calling natif des LLMs.
- La boucle d'exécution agentique (Agent Loop).
- Gestion des erreurs et de la persistance de l'état dans un tour de parole.

5. Outils et technologies à découvrir

- Fonctionnalités natives de Tool Calling des API OpenAI / Anthropic / Gemini.
- LangChain (composants de base Core/Expression Language) ou exécution pure (Vanilla Python/JS).
- Instructor / Pydantic (pour structurer les entrées/sorties de fonctions).

6. Prérequis pour aborder la phase

- Validation de la Phase 1 (maîtrise des appels API LLM et des prompts).

7. Types d'exercices à réaliser

- Implémentation d'une fonction simple de calculatrice connectée à un LLM en mode tool calling.
- Création d'une boucle agentique basique sans framework (from scratch) en Python/JS.
- Gestion d'un appel d'outil retournant une erreur gérée par le LLM.
- Implémentation de schémas de validation stricts pour les arguments d'outils.

8. Projet capstone de fin de phase

- Création d'un agent conversationnel capable de répondre à des questions factuelles complexes en combinant l'accès à une base de données locale simulée via des outils personnalisés et l'interrogation d'une API externe (ex: météo ou conversion de devises).

9. Critères de validation pour passer à la phase suivante

- Compréhension intime du fonctionnement d'une boucle agentique (sans dépendre aveuglément d'une boîte noire).
- Capacité à concevoir des outils propres documentés (docstrings et schémas JSON) exploitables de manière fiable par un LLM.
- Résolution réussie de scénarios où l'agent doit enchaîner plusieurs outils pour aboutir à une réponse.

10. Ressources recommandées

- Documentation officielle sur le Function / Tool Calling des principaux fournisseurs.
- Articles fondateurs sur le pattern ReAct (Yao et al.).
- Tutoriels sur Pydantic et la validation de schémas de données.

PHASE 3 : AGENTS AVEC MÉMOIRE ET CONTEXTE

1. Nom de la phase et durée estimée

- Nom : Gestion de la Mémoire et Recherche Augmentée (RAG)
- Durée estimée : 3 semaines

3. Liste des compétences clés à acquérir

- Implémenter différentes stratégies de mémoire (mémoire conversationnelle à court terme, résumé d'historique à long terme).
- Générer des embeddings textuels à l'aide de modèles spécialisés.
- Stocker et interroger des vecteurs dans une base de données dédiée (Vector Store).
- Concevoir et implémenter un pipeline RAG (Retrieval-Augmented Generation) complet.

4. Liste des concepts théoriques à comprendre

- Espaces vectoriels et similarité cosinus.
- Stratégies de découpage de texte (Text Chunking et chevauchement/overlap).
- RAG naïf vs RAG avancé (hybride, re-ranking, query transformation).
- Gestion dynamique de la fenêtre de contexte et élagage de l'historique (Memory Pruning).

5. Outils et technologies à découvrir

- ChromaDB / FAISS / Qdrant / LanceDB (Bases de données vectorielles open source).
- Embeddings API (OpenAI text-embedding-3, modèles open source Hugging Face / Sentence-Transformers).
- Bibliothèques de manipulation de documents (PyPDF, LangChain document loaders).

6. Prérequis pour aborder la phase

- Validation de la Phase 2 (maîtrise de la boucle agentique et du tool calling).

7. Types d'exercices à réaliser

- Script de découpage (chunking) d'un document texte long avec différents paramètres.
- Génération d'embeddings et stockage dans une base vectorielle locale.
- Implémentation d'un outil de recherche sémantique que l'agent peut appeler (Vector Search Tool).
- Ajout d'un système de gestion de l'historique de conversation persistant en base de données.

8. Projet capstone de fin de phase

- Développement d'un agent expert sur une documentation technique interne : l'agent indexe un ensemble de fichiers PDF locaux, stocke les embeddings, et utilise un outil de recherche sémantique combiné à sa boucle de raisonnement pour répondre précisément aux questions des utilisateurs en citant ses sources.

9. Critères de validation pour passer à la phase suivante

- Capacité à évaluer la pertinence des résultats récupérés par recherche sémantique (precision/recall du retrieval).
- Maîtrise de l'intégration de la mémoire conversationnelle dans une architecture agentique.
- Compréhension des limites des fenêtres de contexte et des stratégies de mitigation.

10. Ressources recommandées

- Documentation des bases de données vectorielles open source.
- Littérature académique et technique sur le RAG avancé (blogs de Pinecone, LlamaIndex, etc.).
- Cours sur les embeddings et la représentation géométrique du texte.

PHASE 4 : AGENTS MULTI-OUTILS ET ORCHESTRATION

1. Nom de la phase et durée estimée

- Nom : Orchestration Avancée et Standardisation (MCP)
- Durée estimée : 3 semaines

2. Objectif global de la phase

- Étendre les capacités des agents à interagir avec des environnements complexes, hétérogènes et distants grâce à des architectures de protocoles standardisés et des schémas de planification avancés.

3. Liste des compétences clés à acquérir

- Concevoir et implémenter des serveurs et clients basés sur le Model Context Protocol (MCP).
- Mettre en place des stratégies de planification complexes (Plan-and-Solve, réflexion autonome).
- Gérer l'exécution concurrente ou séquentielle d'outils multiples par un agent.
- Assurer la robustesse des flux d'exécution face aux pannes d'outils (gestion des retries, auto-correction).

4. Liste des concepts théoriques à comprendre

- Le Model Context Protocol (architecture hôte/client/serveur d'outils).
- Planification dynamique et décomposition de tâches complexes (Task Decomposition).
- Gestion de l'état partagé et des contextes d'exécution isolés (Sandboxing).
- Évaluation de la trajectoire de l'agent (Trajectory Evaluation).

5. Outils et technologies à découvrir

- Model Context Protocol (SDKs officiels MCP).
- Docker (pour isoler l'exécution des outils si nécessaire).
- Outils de tracing et d'observabilité (LangSmith, Phoenix, Arize).

6. Prérequis pour aborder la phase

- Validation de la Phase 3 (maîtrise du RAG et de la gestion de contexte).

7. Types d'exercices à réaliser

- Création d'un serveur MCP personnalisé exposant des outils locaux (ex: système de fichiers, exécution de scripts).
- Connexion d'un agent à ce serveur MCP via un client standard.
- Implémentation d'un pattern de planification où l'agent rédige un plan d'action étape par étape avant d'exécuter les outils.
- Intégration d'un outil de traçage pour visualiser l'arbre d'exécution des appels de l'agent.

8. Projet capstone de fin de phase

- Conception d'un agent DevOps autonome capable de se connecter via MCP à un environnement de staging local, de planifier la résolution d'un problème technique (ex: analyse de logs d'erreur, exécution de commandes de test isolées, correction de code et validation), tout en traçant l'intégralité de sa démarche.

9. Critères de validation pour passer à la phase suivante

- Maîtrise de la mise en œuvre et de la consommation de serveurs MCP.

- Capacité à concevoir des agents capables de décomposer de grands objectifs en sous-tâches gérables.
- Utilisation efficace d'outils de traçage pour déboguer les boucles de raisonnement complexes.

10. Ressources recommandées

- Spécifications officielles et documentation du Model Context Protocol (MCP).
- Papiers de recherche sur les architectures de planification d'agents autonomes.
- Documentation des plateformes d'observabilité LLM.

PHASE 5 : AGENTS AUTONOMES ET MULTI-AGENTS

1. Nom de la phase et durée estimée

- Nom : Systèmes Multi-Agents et Frameworks d'Orchestration Avancés
- Durée estimée : 4 semaines

2. Objectif global de la phase

- Concevoir, orchestrer et coordonner des systèmes complexes impliquant plusieurs agents spécialisés collaborant pour résoudre des tâches de grande envergure.

3. Liste des compétences clés à acquérir

- Modéliser des rôles, des personas et des compétences distinctes pour différents agents.
- Orchestrer des workflows séquentiels, hiérarchiques ou basés sur des graphes d'état entre agents.
- Gérer les communications inter-agents (messages, tableaux de bord partagés, délégations).
- Résoudre les conflits et gérer les boucles de rétroaction (peer review) entre agents.

4. Liste des concepts théoriques à comprendre

- Architectures multi-agents (collaborative vs compétitive).

- State Machines (machines à états) appliquées aux workflows LLM.
- Modèles de délégation de tâches et de supervision hiérarchique.
- Gestion du consensus et de la validation croisée (cross-validation).

5. Outils et technologies à découvrir

- LangGraph (pour la construction de graphes d'agents étatiques).
- CrewAI / AutoGen (Frameworks de haut niveau multi-agents).
- Bee Agent Framework ou équivalents open source modulaires.

6. Prérequis pour aborder la phase

- Validation de la Phase 4 (maîtrise du tool calling, des plans et du traçage).

7. Types d'exercices à réaliser

- Création d'un mini-système à deux agents (un rédacteur et un correcteur) échangeant des messages.
- Implémentation d'un workflow en graphe avec LangGraph incluant des conditions de routage dynamiques.
- Configuration d'une équipe hiérarchique (un manager et deux agents exécutants) avec CrewAI ou AutoGen.
- Gestion de la mémoire partagée entre plusieurs agents distincts.

8. Projet capstone de fin de phase

- Développement d'une agence virtuelle de rédaction de contenu de recherche : un agent "Chercheur" collecte des informations via le web/RAG, un agent "Rédacteur" produit un article structuré, un agent "Relecteur" critique le contenu selon des critères stricts, et un agent "Éditeur" valide la version finale, le tout orchestré via une machine à états robuste.

9. Critères de validation pour passer à la phase suivante

- Capacité à concevoir et déboguer un système multi-agents complexe sans pertes de contexte.
- Maîtrise des concepts de graphes d'état et de gestion des transitions conditionnelles.

- Compréhension des cas d'usage où le multi-agent apporte une réelle valeur ajoutée par rapport à un agent unique.

10. Ressources recommandées

- Documentation officielle de LangGraph, CrewAI et AutoGen.
- Articles de recherche sur les systèmes multi-agents coopératifs.
- Retours d'expérience et cas d'études industriels sur l'architecture multi-agents.

PHASE 6 : ÉVALUATION, SÉCURITÉ, GARDE-FOUS ET DÉPLOIEMENT

1. Nom de la phase et durée estimée

- Nom : Production, Évaluation rigoureuse et Sécurité des Agents
- Durée estimée : 3 semaines

2. Objectif global de la phase

- Sécuriser les applications agentiques contre les vulnérabilités, évaluer systématiquement leurs performances et déployer des solutions prêtes pour la production.

3. Liste des compétences clés à acquérir

- Mettre en place des pipelines d'évaluation automatisée (frameworks LLM-as-a-judge).
- Identifier et contrer les vulnérabilités de sécurité courantes (Prompt Injection, Jailbreaking, fuite de données, exécution de code non sécurisée).
- Implémenter des garde-fous (guardrails) d'entrée et de sortie.
- Conteneuriser et déployer un agent sur une infrastructure cloud ou en serverless.

4. Liste des concepts théoriques à comprendre

- L'OWASP Top 10 pour les applications LLM (Prompt Injection, Insecure Output Handling, etc.).
- Évaluation basée sur des benchmarks et des jeux de données de test (Golden Datasets).
- Guardrails, filtrage de contenu et détection d'hallucinations.

- Stratégies de résilience (rate limiting, circuits breakers, fallbacks de modèles).

5. Outils et technologies à découvrir

- NeMo Guardrails / Llama Guard (Sécurité et filtrage).
- DeepEval / Ragas / Promptfoo (Outils d'évaluation automatisée).
- Docker / Kubernetes / Cloud Run / AWS Lambda (Déploiement).
- LangSmith / Phoenix (pour le monitoring en production).

6. Prérequis pour aborder la phase

- Validation de la Phase 5 (maîtrise des architectures d'agents et multi-agents).

7. Types d'exercices à réaliser

- Implémentation d'un filtre de garde-fou bloquant les tentatives d'injection de prompt malveillantes.
- Création d'un jeu de données de test (Golden Dataset) et évaluation automatisée d'un agent avec un framework dédié.
- Conteneurisation d'un agent simple via Docker.
- Mise en place d'un mécanisme de secours (fallback) vers un modèle plus petit ou une réponse statique en cas de timeout.

8. Projet capstone de fin de phase

- Audit de sécurité et industrialisation complète d'un agent développé lors des phases précédentes : intégration de garde-fous stricts contre les injections de prompts, mise en place d'un pipeline d'évaluation automatisée de la qualité des réponses sur un jeu de test de référence, et déploiement de l'agent dans un conteneur Docker sécurisé accessible via une API de production.

9. Critères de validation pour passer à la phase suivante

- Capacité à concevoir une stratégie d'évaluation quantitative des performances d'un agent.
- Maîtrise des principales failles de sécurité spécifiques aux LLMs et mise en œuvre de contre-mesures efficaces.
- Aptitude à packager et déployer un système agentique en environnement conteneurisé.

10. Ressources recommandées

- Documentation de l'OWASP Top 10 for LLM Applications.
- Guides et documentations des frameworks d'évaluation et de sécurité (Ragas, NeMo Guardrails).
- Littérature sur l'ingénierie de la fiabilité des systèmes IA (AI Reliability Engineering).

PHASE 7 : EXPERTISE

1. Nom de la phase et durée estimée

- Nom : Optimisation Avancée, Fine-Tuning et Modèles Open Source
- Durée estimée : 4 semaines

2. Objectif global de la phase

- Atteindre le niveau d'expertise en optimisant les coûts, la latence et les performances des agents par le choix de modèles open source, le fine-tuning spécialisé et l'optimisation des architectures d'inférence.

3. Liste des compétences clés à acquérir

- Sélectionner, télécharger et exécuter des modèles open source en local ou sur infrastructure dédiée.
- Préparer des jeux de données d'entraînement et réaliser du fine-tuning (Supervised Fine-Tuning) pour des tâches spécifiques d'agents.
- Optimiser la vitesse d'inférence (quantification, moteurs d'inférence haute performance).
- Concevoir des architectures d'agents hybrides combinant petits modèles locaux spécialisés et grands modèles de raisonnement généraux.

4. Liste des concepts théoriques à comprendre

- Techniques de quantification (GGUF, AWQ, GPTQ, FP8).
- Fine-Tuning supervisé (SFT) et méthodes d'alignement (DPO, ORPO).
- Inférence locale vs inférence cloud managée.
- Distillation de modèles et spécialisation pour le tool calling.

5. Outils et technologies à découvrir

- Ollama / LM Studio (pour exécuter des modèles open source en local).
- vLLM / llama.cpp (Moteurs d'inférence haute performance).
- Hugging Face Transformers / PEFT / TRL (pour l'entraînement et le fine-tuning).
- Unsloth (pour un fine-tuning efficace et rapide).

6. Prérequis pour aborder la phase

- Validation de la Phase 6 (maîtrise de la production, de la sécurité et du déploiement).

7. Types d'exercices à réaliser

- Exécution et test d'un modèle open source local (ex: Llama ou Mistral) via Ollama ou llama.cpp.
- Préparation d'un jeu de données au format JSONL pour l'entraînement au tool calling d'un petit modèle.
- Réalisation d'un fine-tuning léger (QLoRA) sur un modèle open source pour spécialiser son comportement d'agent.
- Comparaison comparative des performances de latence et de coût entre un grand modèle cloud et un modèle local quantifié.

8. Projet capstone de fin de phase

- Refonte complète d'un système d'agent multi-agents complexe en remplaçant les appels systématiques aux API propriétaires cloud par un mix hybride : utilisation de petits modèles open source finement ajustés pour les tâches de classification et d'appel d'outils exécutés localement via un moteur d'inférence optimisé, réservant les grands modèles propriétaires uniquement aux tâches complexes de raisonnement de haut niveau.

9. Critères de validation pour passer au stade d'expert

- Capacité avérée à déployer, orchestrer et optimiser des modèles open source pour des cas d'usage agentiques spécifiques.
- Maîtrise des compromis entre coût, latence, taille du modèle et performance de raisonnement.

- Autonomie complète dans la conception, la sécurisation, l'évaluation et l'optimisation d'architectures d'agents de pointe.

10. Ressources recommandées

- Documentation officielle de Hugging Face, vLLM et Ollama.
- Papiers de recherche récents sur l'optimisation des LLMs et le fine-tuning orienté agents.
- Littérature technique de pointe sur l'inférence locale et la quantification.